

FITFLOW

Documentação API

Trabalho Integrador - Técnico em Informática

Nome do aluno completo
Orientador: Prof. Fernando Paim Lima
Coorientador: Prof. Roger Santos Ferreira

FORMIGA - MG

2026

Documentação da API - FitFlow

Autenticação: Bearer Token (Sanctum)

Formato de Retorno: JSON

1 .Endpoints de Autenticação

1.1. Login de Usuário

- **Método:** POST
- **Rota:** /login
- **Descrição:** Autentica um usuário no sistema (usado pelo aplicativo Flutter). Verifica e-mail, senha e se o usuário é administrador.
- **Corpo da Requisição:**
- JSON

```
{  
  "email": "usuario@exemplo.com",  
  "password": "senha_secreta"  
}
```

(Os campos *email* e *password* são obrigatórios)

- **Respostas:**
 - **Sucesso (200 OK):**
 - JSON
- ```
{
 "token": "1|abc123def456...",
 "user": {
 "id": 1,
 "name": "Nome do Usuário",
 "user_name": "nickname",
 "email": "usuario@exemplo.com",
 "idade": 25,
 "cidadeMora": "Formiga",
 "urlImage": "https://..."
 }
}
```
-

(O objeto *user* é formatado pela classe *UserResource*).

- **Erro - Credenciais Inválidas (401 Unauthorized):**
- JSON

```
{
 "message": "Credenciais inválidas"
}
```

- **Erro - E-mail não verificado (403 Forbidden):**
- JSON

```
{
 "message": "Email não verificado!"
}
```

- **Erro - Acesso de Administrador Negado (403 Forbidden):**
- JSON

```
{
 "message": "Acesso negado para Administradores",
 "isAdmin": true
}
```

## 1.2. Obter Dados do Usuário Logado

- **Método:** GET
- **Rota:** /me
- **Middleware:** auth:sanctum, verified
- **Header Obrigatório:** Authorization: Bearer {token}
- **Descrição:** Retorna os dados do usuário autenticado no momento.
- **Resposta de Sucesso (200 OK):** Retorna os mesmos campos do *UserResource* exibidos no login.

## 1.3. Logout

- **Método:** POST
- **Rota:** /logout
- **Middleware:** auth:sanctum, verified

- **Header Obrigatório:** `Authorization: Bearer {token}`
- **Descrição:** Revoga o token de acesso atual do usuário.
- **Resposta de Sucesso (204 No Content):** Retorna vazio.

## 2. Endpoints de Dados do App

### 2.1. Buscar Dados (Fichas e Exercícios) do Usuário

- **Método:** `GET`
- **Rota:** `/pegaDados/{id}`
- **Middleware:** `auth:sanctum, verified`
- **Parâmetros de Rota:** `id` (String - ID do usuário)
- **Descrição:** Busca um usuário pelo ID e retorna todas as suas fichas de treino, juntamente com os exercícios associados a cada ficha.
- **Respostas:**
  - **Sucesso (200 OK):**
  - JSON

```
{
 "fichas": [
 {
 "id": 1,
 "name": "Treino A",
 "data_inicio": "01/10/2026",
 "data_fim": "31/10/2026",
 "descricao": "Foco em membros superiores"
 }
],
 "fichaExercicio": [
 {
 "id": 10,
 "treino": "Nome do Treino ",
 "ficha_id": 1,
 "dias_semana": "Seg,Qua,Sex",
 "ordem": 1,
 "qntdSeries": 4,
 "qntdRep": "12",
 "peso": "20kg",
 "descricao": "Supino reto",
 "descanso": "60s"
 }
]
}
```

```
]
}
```

(Os dados são formatados pelas classes *FichasResource* e *ExercicioResource*).

(Note que há uma união do model *exercicio* com *ficha* gerando o *fichaExercicio*).

- **Erro de Servidor (500 Internal Server Error):**
- JSON

```
{
 "message": "algo deu errado (no php authController.pegaDados)"
}
```

### 3. Serviços Internos (CloudinaryService)

Embora estes não sejam rotas diretas (como definido em *api.php*), eles detalham como o back-end processa as imagens e devem ser mencionados na arquitetura.

#### 3.1. Upload de Imagem

- **Método Interno:** *uploadImage(\$file, \$folder = 'perfis')*
- **Descrição:** O Laravel recebe um arquivo e o envia para a API do Cloudinary via requisição *POST* com autenticação (assinatura SHA1 e Timestamp).
- **Retorno Esperado:** Em caso de sucesso, o serviço retorna a URL segura (*secure\_url*) da imagem gerada pelo Cloudinary. Caso contrário, retorna *null*.

#### 3.2. Deletar Imagem

- **Método Interno:** *deletelImage(\$imageUrl)*
- **Descrição:** Extrai o *public\_id* da URL da imagem e envia uma requisição *POST* para a API do Cloudinary para remover o arquivo correspondente.